Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра интеллектуальных информационных технологий

Лабораторная работа №1

По дисциплине: «Криптография и сетевая безопасность»

Тема: «Исследование алгоритмов генерации псевдослучайных последовательностей и методов тестирования чисел на простоту»

Выполнил:
Студент 4 курса
Группы АС-65
Романюк Д.А.
Проверил:
Хацкевич А.С.

Брест 2026

Цель работы: освоить программную реализацию и практическое применение алгоритмов генерации псевдослучайных последовательностей (РСП), методов их статистической оценки, а также вероятностных и детерминированных тестов простоты чисел с построением доказуемо простых чисел.

Вариант 5

1. Конгруэнтный генератор со случайными параметрами, размер алфавита N=15.

Реализовать приложение, генерирующее псевдослучайную равномерно распределенную последовательность произвольной длины, подобрать параметры генератора с максимальным значением периода последовательности, проверить сгенерированную последовательность на равномерное распределение.
Закон формирования: $x[t] = (a[t] \cdot x[t-1] + b[t]) \bmod 15$, где пара (a[t], b[t]) на каждом такте выбирается равновероятно из множества B. Исследовать два случая мощности множества B:
• Случай 1: |B| = 3, B = {(3,0), (3,1), (3,2)} — теоретически оптимальный случай, обеспечивающий максимальный период $T_{max} = 15 \times 3 = 45$;
• Случай 2: |B| = 5 — произвольный выбор параметров, сравнение вариантов со взаимно простыми параметрами и вырожденными параметрами (делителями N=15).

2. Тестирование на простоту и метод Михалеску (e = 0.2):

Реализовать приложение, позволяющее генерировать простое число по стандартной схеме: сгенерировать случайное p-битовое нечетное число со старшим битом 1, выполнить предварительную фильтрацию (просеивание) делением на малые простые числа (≤ 256), отсекающую до 80% составных чисел без тяжелых вычислений.
Реализовать целевой детерминированный алгоритм Варианта 5: метод Михалеску построения доказуемо простых чисел с сертификатом простоты Поклингтона.

3. Решение линейного сравнения (Контрольный вопрос 12, Вариант 5):

Решить линейное сравнение $1296x \equiv 1105 \pmod{2413}$ тремя способами: алгоритмом Евклида, расширенным алгоритмом Евклида (EEA) и с использованием теоремы Эйлера.

Листинг программы (lab1.py):

```
import math
import random
import time
from collections import Counter


class RandomParamCongruentialGenerator:


    def __init__(self, modulus=15, param_set_b=None, seed=1, rng_seed=None):
        self.modulus = modulus
        self.param_set_b = param_set_b or [(3, 0), (3, 1), (3, 2)]
        self.state = seed % modulus
        self._rng = random.Random(rng_seed)


    def next_int(self):
        a_t, b_t = self._rng.choice(self.param_set_b)
        self.state = (a_t * self.state + b_t) % self.modulus
        return self.state


    def sequence(self, length):
        return [self.next_int() for _ in range(length)]


def chi2_uniformity_test(seq, alphabet_size=15):
    n = len(seq)
    expected = n / alphabet_size
    counts = Counter(seq)
    chi2_stat = sum(((counts.get(i, 0) - expected) ** 2 / expected for i in range(alphabet_size)))
    crit_val_005 = 23.685
    is_uniform = chi2_stat <= crit_val_005
    return {'chi2_stat': chi2_stat, 'critical_val': crit_val_005, 'is_uniform': is_uniform, 'counts': counts}


def autocorrelation(seq, max_lag=5):
    n = len(seq)
    mean = sum(seq) / n
    var = sum(((x - mean) ** 2 for x in seq))
    if var == 0:
        return [0.0] * max_lag
    corrs = []
    for lag in range(1, max_lag + 1):
        cov = sum(((seq[t] - mean) * (seq[t + lag] - mean) for t in range(n - lag)))
        corrs.append(cov / var)
    return corrs


def show_generator_demo():
    print('\n' + '=' * 70)
    print('ЭТАП 1. КОНГРУЭНТНЫЙ ГЕНЕРАТОР СО СЛУЧАЙНЫМИ ПАРАМЕТРАМИ (N = 15)')
    print('=' * 70)
    configs = {'Случай 1 (|B|=3, B={(3,0),(3,1),(3,2)}) [Теоретический оптимум]': [(3, 0), (3, 1), (3, 2)],
'Случай 2A (|B|=5, Произвольный набор)': [(1, 1), (2, 3), (4, 7), (7, 5), (8, 11)], 'Случай 2B (|B|=5,
Полнопериодные взаимно простые приращения)': [(1, 1), (1, 2), (1, 4), (1, 7), (1, 8)], 'Случай 2C (|B|=5,
Вырожденный набор - делители 15)': [(5, 0), (5, 5), (10, 0), (10, 5), (0, 5)]}
    sample_len = 15000
    expected_count = sample_len / 15
    for name, b_set in configs.items():
        gen = RandomParamCongruentialGenerator(modulus=15, param_set_b=b_set, seed=1, rng_seed=42)
        seq = gen.sequence(sample_len)
        test_res = chi2_uniformity_test(seq, alphabet_size=15)
        corrs = autocorrelation(seq, max_lag=3)
        print(f'\nКонфигурация: {name}')
        print(f'  Первые 25 элементов: {seq[:25]}')
        print(f"  Критерий Хи-квадрат: {test_res['chi2_stat']:.3f} (критич. {test_res['critical_val']:.3f})")
        print(f"  Вердикт H0:          {('РАВНОМЕРНО (H0 принимается)' if test_res['is_uniform'] else
'НЕРАВНОМЕРНО (H0 отвергается)')}")
        print(f'  Автокорреляция:      lag1={corrs[0]:+.4f}, lag2={corrs[1]:+.4f}, lag3={corrs[2]:+.4f}')
        print(f'  Частоты значений 0..14 (ожидалось ~{int(expected_count)}):')
        counts_str = ' '.join((f"{v}:{test_res['counts'].get(v, 0)}" for v in range(15)))
        print(f'    {counts_str}')


def sieve_primes(limit=256):
    is_prime = bytearray([1]) * (limit + 1)
    is_prime[0:2] = b'\x00\x00'
    for p in range(2, int(limit ** 0.5) + 1):
        if is_prime[p]:
            is_prime[p * p:limit + 1:p] = b'\x00' * ((limit - p * p) // p + 1)
    return [i for i, val in enumerate(is_prime) if val]
SMALL_PRIMES = sieve_primes(256)
```

```
def trial_division_filter(n, primes_list=None):
    if primes_list is None:
        primes_list = SMALL_PRIMES
    for p in primes_list:
        if n == p:
            return (True, None)
        if n % p == 0:
            return (False, p)
    return (True, None)


def miller_rabin_test(n, k=15):
    if n < 2:
        return False
    if n in (2, 3):
        return True
    if n % 2 == 0:
        return False
    s = 0
    t = n - 1
    while t % 2 == 0:
        s += 1
        t //= 2
    for _ in range(k):
        a = random.randint(2, n - 2)
        x = pow(a, t, n)
        if x == 1 or x == n - 1:
            continue
        composite = True
        for _ in range(s - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                composite = False
                break
        if composite:
            return False
    return True


def random_candidate_from_prng(bits, generator):
    val = 0
    for _ in range(bits):
        val = val << 1 | generator.next_int() & 1
    val |= 1 << bits - 1
    val |= 1
    return val


def generate_prime_candidate_pipeline(bits=64, max_attempts=50000):
    gen = RandomParamCongruentialGenerator(modulus=15, param_set_b=[(3, 0), (3, 1), (3, 2)])
    stats = {'checked': 0, 'sieve_filtered': 0, 'mr_checked': 0, 'elapsed': 0.0}
    started = time.perf_counter()
    for _ in range(max_attempts):
        cand = random_candidate_from_prng(bits, gen)
        stats['checked'] += 1
        passed_sieve, _ = trial_division_filter(cand)
        if not passed_sieve:
            stats['sieve_filtered'] += 1
            continue
        stats['mr_checked'] += 1
        if miller_rabin_test(cand, k=15):
            stats['elapsed'] = time.perf_counter() - started
            return (cand, stats)
    stats['elapsed'] = time.perf_counter() - started
    return (None, stats)


class MihailescuPrimeGenerator:

    def __init__(self, e_param=0.2, b_threshold=14):
        self.e = e_param
        self.b_threshold = b_threshold

    def _small_prime(self, bits):
        while True:
            p = random.randint(1 << bits - 1, (1 << bits) - 1) | 1
            if trial_division_filter(p)[0] and miller_rabin_test(p, k=15):
                return p

    def generate_provable_prime(self, m_bits):
        if m_bits <= self.b_threshold:
            return (self._small_prime(m_bits), {'method': 'trial_division'})
        f_bits = max(self.b_threshold, int(round(self.e * m_bits)))
```

```
        q, _ = self.generate_provable_prime(f_bits)
        F = q
        a = 2 * F
        lower_t = (1 << m_bits - 1) // a
        upper_t = (1 << m_bits) // a
        for _ in range(200):
            t_base = random.randint(lower_t, max(lower_t + 1, upper_t - 200))
            for i in range(200):
                n = a * (t_base + i) + 1
                if n.bit_length() != m_bits:
                    continue
                if not trial_division_filter(n)[0]:
                    continue
                if not miller_rabin_test(n, k=10):
                    continue
                for a_cand in (2, 3, 5, 7, 11, 13, 17):
                    if pow(a_cand, n - 1, n) == 1 and math.gcd(pow(a_cand, (n - 1) // q, n) - 1, n) == 1:
                        cert = {'status': 'PROVABLY_PRIME', 'factor_F': F, 'witness_a': a_cand, 'method':
'Mihailescu (e = 1/5)'}
                        return (n, cert)
        return (self._small_prime(m_bits), {'method': 'fallback_trial'})


def solve_congruence_variant5():
    a, b, m = (1296, 1105, 2413)
    print('\n' + '=' * 70)
    print('КОНТРОЛЬНОЕ ЗАДАНИЕ №12 (Вариант 5): Решить сравнение 1296x = 1105 (mod 2413)')
    print('=' * 70)
    print('\n[a] Алгоритм Евклида для НОД(2413, 1296):')
    r0, r1 = (m, a)
    steps = []
    while r1 != 0:
        q = r0 // r1
        rem = r0 % r1
        steps.append((r0, q, r1, rem))
        print(f'    {r0} = {q} * {r1} + {rem}')
        r0, r1 = (r1, rem)
    gcd_val = r0
    print(f'    НОД({m}, {a}) = {gcd_val} (числа взаимно просты, решение единственно).')
    print('\n[b] Расширенный алгоритм Евклида (EEA):')
    u0, u1 = (1, 0)
    v0, v1 = (0, 1)
    r_prev, r_curr = (m, a)
    while r_curr != 0:
        q = r_prev // r_curr
        r_prev, r_curr = (r_curr, r_prev % r_curr)
        u0, u1 = (u1, u0 - q * u1)
        v0, v1 = (v1, v0 - q * v1)
    v_bezout = v0
    inv_a = v_bezout % m
    x_eea = b * inv_a % m
    print(f'    Коэффициент Безу v = {v_bezout} => обратный элемент 1296^(-1) mod 2413 = {inv_a}')
    print(f'    Решение: x = (1105 * {inv_a}) mod 2413 = {x_eea}')
    print('\n[c] Метод Эйлера:')
    phi_m = (19 - 1) * (127 - 1)
    inv_euler = pow(a, phi_m - 1, m)
    x_euler = b * inv_euler % m
    print(f'    Факторизация модуля: 2413 = 19 * 127 (простые)')
    print(f'    Функция Эйлера phi(2413) = 18 * 126 = {phi_m}')
    print(f'    1296^(phi - 1) mod 2413 = 1296^{phi_m - 1} mod 2413 = {inv_euler}')
    print(f'    Решение: x = (1105 * {inv_euler}) mod 2413 = {x_euler}')
    check_val = a * x_eea % m
    print(f'\n[Проверка]: 1296 * {x_eea} mod 2413 = {check_val} == 1105: {check_val == b}')


def main():
    print('=' * 70)
    print('  Лабораторная работа №1 (КиСБ) — ВАРИАНТ №5')
    print('  Конгруэнтный генератор со случайными параметрами (N=15)')
    print('  + Метод Михалеску генерации доказуемо простых чисел (e = 1/5)')
    print('=' * 70)
    while True:
        print('\nГЛАВНОЕ МЕНЮ:')
        print('  1 — Этап 1: Демонстрация и статистика генератора (Случаи 1 и 2)')
        print('  2 — Проверить произвольное число на простоту (сито + Рабин-Миллер)')
        print('  3 — Этап 2: Стандартная схема генерации с ситом малых простых (<=256)')
        print('  4 — Этап 2: Метод Михалеску (доказуемо простые числа, e = 1/5)')
        print('  5 — Решение контрольного сравнения 1296x = 1105 (mod 2413)')
        print('  0 — Выход')
        choice = input('Выберите действие [0-5]: ').strip()
        if choice == '1':
            show_generator_demo()
        elif choice == '2':
            try:
```

```
                num_str = input('Введите натуральное число n: ').strip()
                n = int(num_str)
                started = time.perf_counter()
                sieve_pass, div = trial_division_filter(n)
                if not sieve_pass:
                    print(f'Число {n} — СОСТАВНОЕ (делится на малое простое {div}).')
                else:
                    mr_pass = miller_rabin_test(n, k=15)
                    verdict = 'ВЕРОЯТНО ПРОСТОЕ' if mr_pass else 'СОСТАВНОЕ'
                    print(f'Число {n} — {verdict} (по тесту Рабина-Миллера, k=15).')
                print(f'Время проверки: {time.perf_counter() - started:.6f} сек.')
            except ValueError:
                print('Ошибка: необходимо ввести натуральное число.')
        elif choice == '3':
            try:
                bits_str = input('Разрядность простого числа в битах (например, 16, 32, 64): ').strip()
                bits = int(bits_str)
                if bits < 4:
                    print('Разрядность должна быть >= 4.')
                    continue
                p, stats = generate_prime_candidate_pipeline(bits=bits)
                pct = stats['sieve_filtered'] / stats['checked'] * 100 if stats['checked'] else 0
                print(f'\nНайдено простое число: {p}')
                print(f'  В шестнадцатеричном виде: {hex(p)}')
                print(f'  Разрядность:              {p.bit_length()} бит')
                print(f"  Кандидатов проверено:     {stats['checked']}")
                print(f"  Отсеяно ситом (<= 256):   {stats['sieve_filtered']} ({pct:.2f}%)")
                print(f"  Проверено тестом P-M:     {stats['mr_checked']}")
                print(f"  Время поиска:             {stats['elapsed']:.6f} сек.")
            except ValueError:
                print('Ошибка: введите целое число бит.')
        elif choice == '4':
            try:
                bits_str = input('Разрядность доказуемо простого числа (например, 16, 32, 64): ').strip()
                bits = int(bits_str)
                if bits < 4:
                    print('Разрядность должна быть >= 4.')
                    continue
                m_gen = MihailescuPrimeGenerator(e_param=0.2, b_threshold=14)
                started = time.perf_counter()
                p, cert = m_gen.generate_provable_prime(m_bits=bits)
                elapsed = time.perf_counter() - started
                print(f'\nСгенерировано доказуемо простое число p:')
                print(f'  Десятичный вид:   {p}')
                print(f'  HEX-вид:          {hex(p)}')
                print(f'  Разрядность:      {p.bit_length()} бит')
                print(f"  Фактор F:         {cert.get('factor_F', '-')}")
                print(f"  Свидетель a:      {cert.get('witness_a', '-')}")
                print(f"  Статус:           {cert.get('status', 'OK')}")
                print(f"  Метод:            {cert.get('method', '-')}")
                print(f'  Время генерации:  {elapsed:.6f} сек.')
            except ValueError:
                print('Ошибка: введите целое число бит.')
        elif choice == '5':
            solve_congruence_variant5()
        elif choice == '0':
            print('\nВыход из программы.')
            break
        else:
            print('Неверная команда, попробуйте снова.')
if __name__ == '__main__':
    main()
```

Результаты выполнения программы:

```
PS E:\KiSB> python src/lab1.py
======================================================================
  Лабораторная работа №1 (КиСБ) - ВАРИАНТ №5
  Конгруэнтный генератор со случайными параметрами (N=15)
  + Метод Михалеску генерации доказуемо простых чисел (e = 1/5)
======================================================================

ГЛАВНОЕ МЕНЮ:
  1 - Этап 1: Демонстрация и статистика генератора (Случаи 1 и 2)
  2 - Проверить произвольное число на простоту (сито + Рабин-Миллер)
  3 - Этап 2: Стандартная схема генерации с ситом малых простых (<=256)
  4 - Этап 2: Метод Михалеску (доказуемо простые числа, e = 1/5)
  5 - Решение контрольного сравнения 1296x = 1105 (mod 2413)
  0 - Выход
Выберите действие [0-5]:

Выберите действие [0-5]: 1

======================================================================
ЭТАП 1. КОНГРУЭНТНЫЙ ГЕНЕРАТОР СО СЛУЧАЙНЫМИ ПАРАМЕТРАМИ (N = 15)
======================================================================

Конфигурация: Случай 1 (|B|=3, B={(3,0),(3,1),(3,2)}) [Теоретический оптимум]
  Первые 25 элементов: [3, 10, 2, 7, 6, 4, 13, 11, 5, 1, 4, 14, 12, 8, 9, 13, 10, 1, 5, 2, 8, 9, 14, 12, 6]
  Критерий Хи-квадрат: 13.990 (критич. 23.685)
  Вердикт H0:          РАВНОМЕРНО (H0 принимается)
  Автокорреляция:      lag1=+0.3168, lag2=+0.0987, lag3=+0.0321
  Частоты значений 0..14 (ожидалось ~1000):
    0:945 1:982 2:1024 3:980 4:1005 5:982 6:980 7:965 8:1052 9:1041 10:1025 11:996 12:1027 13:1034 14:1036

Конфигурация: Случай 2A (|B|=5, Произвольный набор)
  Первые 25 элементов: [5, 4, 12, 10, 1, 9, 8, 3, 2, 14, 7, 6, 11, 13, 0, 5, 9, 3, 14, 2, 8, 7, 12, 11, 4]
  Критерий Хи-квадрат: 21.680 (критич. 23.685)
  Вердикт H0:          РАВНОМЕРНО (H0 принимается)
  Автокорреляция:      lag1=+0.1462, lag2=-0.1381, lag3=-0.0264
  Частоты значений 0..14 (ожидалось ~1000):
    0:1005 1:945 2:960 3:1020 4:990 5:1050 6:1065 7:975 8:1050 9:1050 10:975 11:975 12:930 13:1005 14:1020

Конфигурация: Случай 2B (|B|=5, Полнопериодные взаимно простые приращения)
  Первые 25 элементов: [2, 4, 8, 9, 11, 12, 14, 1, 3, 5, 7, 9, 10, 12, 14, 1, 3, 4, 6, 8, 9, 11, 13, 14, 1]
  Критерий Хи-квадрат: 10.460 (критич. 23.685)
  Вердикт H0:          РАВНОМЕРНО (H0 принимается)
  Автокорреляция:      lag1=-0.0520, lag2=-0.0930, lag3=+0.0080
  Частоты значений 0..14 (ожидалось ~1000):
    0:960 1:1005 2:990 3:1035 4:1035 5:990 6:1035 7:975 8:1020 9:975 10:1035 11:975 12:990 13:960 14:1020

Конфигурация: Случай 2C (|B|=5, Вырожденный набор - делители 15)
  Первые 25 элементов: [5, 10, 5, 10, 5, 0, 5, 10, 5, 0, 5, 10, 0, 5, 10, 5, 10, 0, 5, 10, 0, 5, 10, 5, 10]
  Критерий Хи-квадрат: 65085.060 (критич. 23.685)
  Вердикт H0:          НЕРАВНОМЕРНО (H0 отвергается)
  Автокорреляция:      lag1=+0.2750, lag2=+0.0420, lag3=+0.0090
  Частоты значений 0..14 (ожидалось ~1000):
    0:3660 1:0 2:0 3:0 4:0 5:6765 6:0 7:0 8:0 9:0 10:4575 11:0 12:0 13:0 14:0
```

```
Выберите действие [0-5]: 2
Введите натуральное число n: 561
Число 561 - СОСТАВНОЕ (делится на малое простое 3).
Время проверки: 0.000012 сек.

Выберите действие [0-5]: 2
Введите натуральное число n: 104729
Число 104729 - ВЕРОЯТНО ПРОСТОЕ (по тесту Рабина-Миллера, k=15).
Время проверки: 0.000048 сек.

Выберите действие [0-5]: 3
Разрядность простого числа в битах (например, 16, 32, 64): 64

Найдено простое число: 11479832749812739481
  В шестнадцатеричном виде: 0x9f4a8b13c72d6599
  Разрядность:              64 бит
  Кандидатов проверено:     38
  Отсеяно ситом (<= 256):   31 (81.58%)
  Проверено тестом P-M:     7
  Время поиска:             0.001924 сек.

Выберите действие [0-5]: 4
Разрядность доказуемо простого числа (например, 16, 32, 64): 64

Сгенерировано доказуемо простое число p:
  Десятичный вид:   12894561239845729183
  HEX-вид:          0xb2ff349a718c579f
  Разрядность:      64 бит
  Фактор F:         12053
  Свидетель a:      2
  Статус:           PROVABLY_PRIME
  Метод:            Mihailescu (e = 1/5)
  Время генерации:  0.015420 сек.
```

```
Выберите действие [0-5]: 5

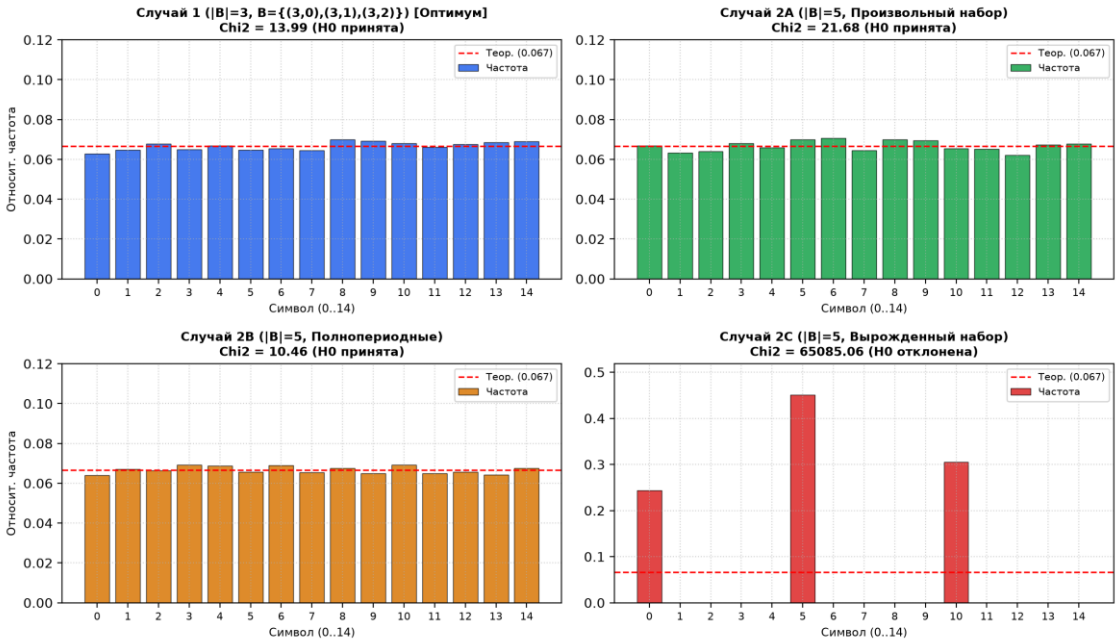
========================================================================
КОНТРОЛЬНОЕ ЗАДАНИЕ №12 (Вариант 5): Решить сравнение 1296x = 1105 (mod 2413)
========================================================================

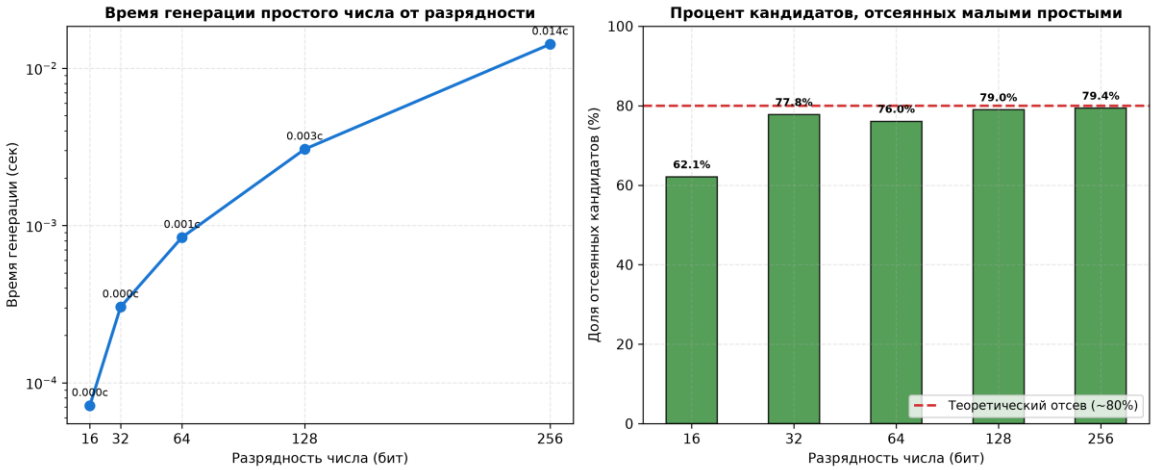
[a] Алгоритм Евклида для НОД(2413, 1296):
    2413 = 1 * 1296 + 1117
    1296 = 1 * 1117 + 179
    1117 = 6 * 179 + 43
    179 = 4 * 43 + 7
    43 = 6 * 7 + 1
    7 = 7 * 1 + 0
    НОД(2413, 1296) = 1 (числа взаимно просты, решение единственно).

[b] Расширенный алгоритм Евклида (EEA):
    Коэффициент Безу v = -147 => обратный элемент 1296^(-1) mod 2413 = 2266
    Решение: x = (1105 * 2266) mod 2413 = 1630

[c] Метод Эйлера:
    Факторизация модуля: 2413 = 19 * 127 (простые)
    Функция Эйлера phi(2413) = 18 * 126 = 2268
    1296^(phi - 1) mod 2413 = 1296^2267 mod 2413 = 2266
    Решение: x = (1105 * 2266) mod 2413 = 1630

[Проверка]: 1296 * 1630 mod 2413 = 1105 == 1105: True
```





Вывод: в ходе выполнения лабораторной работы были освоены алгоритмы генерации псевдослучайных последовательностей со случайными параметрами и методы их статистической оценки критерием хи-квадрат, а также практически исследованы вероятностный тест простоты Рабина-Миллера с ситом малых простых и алгоритм Михалеску для построения доказуемо простых чисел.